

Phased Build Prompts

Twenty staged Codex prompts with acceptance criteria

Phase Prompt Index

Paste one phase at a time. Do not let Codex skip ahead unless the project runs and the acceptance criteria are met.

Phase 0 - Repo Setup

You are working on a new Godot 4 GDScript project called Pacific Firestorm.

Before implementing gameplay, set up the repository structure and project documentation.

Create:

- AGENTS.md
- README.md
- docs/game_design.md
- docs/art_direction.md
- docs/technical_plan.md
- docs/controls.md
- docs/codex_tasks.md
- docs/known_issues.md
- docs/legal_notes.md
- .github/pull_request_template.md

Create the folder structure:

- scenes/
- scripts/
- data/
- assets/
- docs/

Use the structure described in AGENTS.md.

Do not implement gameplay yet.

Do not add copyrighted assets.

Use clear markdown documentation.

Acceptance criteria:

- The repository has the expected folders.
- AGENTS.md gives coding and design rules.
- README.md explains how the project will be opened and run later.
- docs/legal_notes.md states that this is an original game inspired by classic arcade shooters, not a direct clone.

Phase 1 - Godot Skeleton

Implement Phase 1: Godot project skeleton.

Create or update:

- project.godot
- scenes/main/Main.tscn
- scripts/main/main.gd

Main.tscn should contain:

- root Node3D named Main
- WorldRoot Node3D
- Camera3D using orthographic projection
- DirectionalLight3D
- WorldEnvironment if appropriate
- PlaceholderOcean Node3D
- PlayerSpawn Node3D
- EnemyRoot Node3D
- PlayerBulletRoot Node3D
- EnemyBulletRoot Node3D
- EffectsRoot Node3D
- UI CanvasLayer placeholder if practical

main.gd should:

- initialize the scene
- print a startup message
- expose references to important child roots
- not implement advanced gameplay yet

Set up basic input actions in project.godot:

- move_left
- move_right
- move_up
- move_down
- fire
- bomb
- evade
- pause

Acceptance criteria:

- Godot project opens.
- Main.tscn can be set as the main scene.
- Running the scene shows a camera, light, and placeholder world.
- No script parse errors.

Phase 2 - Player Aircraft Placeholder

Implement Phase 2: player aircraft placeholder.

Create:

- scenes/player/Player.tscn
- scripts/player/player.gd

Player.tscn should use:

- CharacterBody3D or Area3D-based movement, whichever is cleaner for this arcade shooter
- a visible placeholder mesh, such as a simple low-poly plane shape or box-based aircraft silhouette
- a child Node3D named GunPoints
- at least two child Node3D gun points: LeftGun and RightGun
- an Area3D or CollisionShape3D for damage hitbox

player.gd should:

- move the player on the X/Z plane
- use input actions move_left, move_right, move_up, move_down
- keep movement arcade-responsive
- clamp the player inside configurable playfield bounds
- not implement realistic flight physics
- expose speed and bounds as exported variables

Update Main.tscn:

- instance Player.tscn at PlayerSpawn

Acceptance criteria:

- The player appears in Main.tscn.
- The player moves in 8 directions.
- The player cannot leave the playfield.
- Movement feels immediate and arcade-like.
- No shooting yet unless trivial hooks are needed.

Phase 3 - Player Shooting and Bullet System

Implement Phase 3: player shooting and bullet system.

Create:

- scenes/bullets/PlayerBullet.tscn
- scripts/bullets/bullet_base.gd
- scripts/player/player_weapon.gd if useful

PlayerBullet.tscn should:

- be an Area3D
- have a visible placeholder tracer/bullet mesh
- have a CollisionShape3D
- move forward/up-screen along the gameplay direction
- destroy or deactivate itself when offscreen

Implement:

- fire cooldown
- dual gun firing from LeftGun and RightGun
- configurable bullet speed

- configurable bullet damage
- a simple bullet lifetime/offscreen cleanup

Prefer simple working implementation first.

If object pooling is not implemented yet, document that pooling will be added later in docs/known_issues.md.

Acceptance criteria:

- Pressing fire shoots bullets.
- Bullets spawn from the player gun points.
- Bullets travel upward/forward.
- Bullets disappear when offscreen.
- No errors when firing repeatedly.

Phase 4 - Enemy Base and First Enemy

Implement Phase 4: enemy base and first enemy.

Create:

- scripts/enemies/enemy_base.gd
- scenes/enemies/EnemyFighter.tscn
- scripts/enemies/movement_patterns.gd

EnemyFighter.tscn should:

- be an Area3D or Node3D with Area3D collision
- have a visible placeholder aircraft mesh
- have a CollisionShape3D
- use enemy_base.gd

enemy_base.gd should support:

- health
- score_value
- movement_speed
- movement_pattern
- take_damage(amount)
- die()
- signal died(score_value)
- cleanup when offscreen

movement_patterns.gd should include at least:

- straight_down
- diagonal_left
- diagonal_right
- sine_wave

Update Main.tscn or main.gd temporarily to spawn one enemy for testing.

Acceptance criteria:

- At least one enemy appears.
- Enemy moves down the screen.
- Enemy cleans up when offscreen.
- Enemy has health and can receive damage through take_damage().

Phase 5 - Bullet/Enemy Collision and Score

Implement Phase 5: bullet-enemy collision and score.

Create:

- scripts/main/game_state.gd
- scripts/ui/hud.gd
- scenes/ui/HUD.tscn

GameState should track:

- score
- lives
- bombs
- current_stage
- game_over state

HUD should display:

- score
- lives
- bombs
- stage name

Connect bullet collision:

- PlayerBullet detects enemy Area3D.
- If collided object has take_damage(), call it.
- Bullet disappears or deactivates after hit.
- Enemy emits died(score_value).
- GameState adds score.
- HUD updates score.

Keep the implementation simple and robust.

Acceptance criteria:

- Player bullets damage enemies.
- Enemy dies when health reaches zero.
- Score increases when enemy dies.
- HUD updates score.
- No duplicate score from one enemy death.

Phase 6 - Player Damage and Lives

Implement Phase 6: player damage and lives.

Add to player:

- take_damage(amount)
- invulnerability window after hit
- visual hit flash placeholder
- signal player_hit
- signal player_destroyed

Update GameState:

- reduce lives when player is hit
- trigger game over when lives reach zero
- reset or respawn player after hit if lives remain

For now, allow collision with enemy aircraft to damage player.
Enemy bullets can be added later.

Acceptance criteria:

- Colliding with an enemy damages the player.
- Player briefly becomes invulnerable after damage.
- Lives decrease on HUD.
- Game over state triggers when lives reach zero.
- Player is not instantly killed multiple times by one collision.

Phase 7 - Enemy Spawner

Implement Phase 7: enemy spawner.

Create:

- scripts/enemies/enemy_spawner.gd
- data/waves/stage_01_waves.json
- data/enemies/enemy_types.json

EnemySpawner should:

- load wave data from JSON
- track stage time
- spawn enemies at defined times
- support enemy_type
- support spawn_x
- support spawn_z
- support movement_pattern
- support formation_id or group_id if simple

Use EnemyFighter.tscn as the first enemy type.

Example wave entry:

```
{
  "time": 1.0,
  "enemy_type": "fighter",
  "spawn_x": -3.0,
  "spawn_z": -16.0,
  "movement": "straight_down"
}
```

Acceptance criteria:

- Stage wave JSON is loaded.
- Enemies spawn according to time.
- At least 10 enemies spawn during a short test wave.

- Enemy movement pattern is applied.
- The system is data-driven, not hard-coded.

Phase 8 - Enemy Bullets

Implement Phase 8: enemy bullet system.

Create:

- scenes/bullets/EnemyBullet.tscn
- script support in bullet_base.gd or separate enemy_bullet.gd

Enemy bullets should:

- be visible and readable
- move downward or toward player depending on mode
- use Area3D collision
- damage player on contact
- clean up offscreen

Update EnemyFighter:

- optional fire_timer
- simple downward shooting
- configurable fire_interval
- configurable bullet_scene

Acceptance criteria:

- Enemies fire bullets.
- Enemy bullets are readable on screen.
- Enemy bullets damage the player.
- Enemy bullets clean up when offscreen.
- Bullet collision does not damage enemies incorrectly.

Phase 9 - Object Pooling

Implement Phase 9: object pooling for bullets and simple effects.

Create:

- scripts/bullets/projectile_pool.gd
- optional scripts/effects/effects_pool.gd

Pooling should support:

- preloading bullet scenes
- reusing inactive bullets
- returning bullets to pool when offscreen or on hit
- avoiding excessive instantiate/free during gameplay

Do not rewrite the entire project.

Make the smallest safe change to replace repeated bullet instantiation with pooling.

Acceptance criteria:

- Player bullets use pool.
- Enemy bullets use pool if practical.
- Existing gameplay remains unchanged.
- No broken collision behaviour.
- Docs explain how the projectile pool works.

Phase 10 - Scrolling Ocean and Environment

Implement Phase 10: scrolling ocean/environment system.

Create:

- scenes/environment/Ocean.tscn
- scripts/environment/ocean_scroller.gd
- scenes/environment/CloudLayer.tscn
- scripts/environment/cloud_scroller.gd
- scripts/environment/environment_spawner.gd

Ocean should:

- provide a placeholder water plane
- scroll visually downward to create flight motion
- support future water material/shader replacement

CloudLayer should:

- spawn simple semi-transparent cloud placeholders
- scroll at a different speed than ocean

- not block gameplay readability

EnvironmentSpawner should eventually support:

- islands
- ships
- smoke columns
- carriers
- wake trails

Acceptance criteria:

- Ocean/background scrolls continuously.
- Player appears to fly forward.
- Clouds or placeholders move at a different speed.
- Gameplay remains readable.

Phase 11 - Power-Ups

Implement Phase 11: power-up system.

Create:

- scenes/powerups/Powerup.tscn
- scripts/powerups/powerup.gd
- data/powerups/powerup_types.json

Power-up types:

- weapon_upgrade
- repair
- bomb
- score_bonus
- rapid_fire_temporary

Powerups should:

- move downward slowly
- have clear visual placeholder
- be collected by player
- apply effect through player or GameState
- clean up offscreen

Update enemies:

- allow optional drop_chance
- allow optional drop_type

Acceptance criteria:

- Enemies can drop power-ups.
- Player can collect power-ups.
- Weapon upgrade changes player firing pattern or bullet count.
- Repair increases lives or health according to current design.
- HUD updates if relevant.

Phase 12 - Weapon Upgrades

Implement Phase 12: weapon upgrade system.

Create or update:

- data/weapons/weapon_types.json
- scripts/player/player_weapon.gd

Weapon levels:

Level 1: dual machine guns
 Level 2: wider dual guns + faster fire
 Level 3: triple shot
 Level 4: spread shot
 Special: bomb/rocket attack

Rules:

- Keep bullets readable.
- Do not flood the screen so much that gameplay becomes unclear.
- Make weapon level configurable.
- Clamp max weapon level.
- Reset weapon level on death only if current game design says so.

Acceptance criteria:

- Collecting weapon_upgrade increases weapon level.
- Different weapon levels visibly change shot pattern.
- Weapon settings come from data or clearly structured code.
- Shooting remains stable and performant.

Phase 13 - Boss System

Implement Phase 13: boss system.

Create:

- scripts/enemies/boss_controller.gd
- scenes/enemies/BossBomber.tscn or BossCarrier.tscn

Boss requirements:

- high health
- visible large placeholder model
- intro movement
- multiple attack phases
- phase changes based on health percentage
- emits boss_defeated signal
- gives large score reward
- triggers mission complete when defeated

Boss phases:

Phase 1: simple forward fire

Phase 2: spread shots

Phase 3: faster fire + side turrets

Phase 4: damaged desperation pattern

Acceptance criteria:

- Boss appears at end of wave.
- Boss has multiple phases.
- Boss can be damaged and defeated.
- Mission complete triggers on boss defeat.
- HUD can display boss health if simple.

Phase 14 - Game States and Menus

Implement Phase 14: game states and menus.

Create:

- scenes/ui/TitleScreen.tscn
- scenes/ui/PauseMenu.tscn
- scenes/ui/GameOverScreen.tscn
- scenes/ui/MissionCompleteScreen.tscn
- scripts/ui/title_screen.gd
- scripts/ui/pause_menu.gd
- scripts/ui/game_over_screen.gd
- scripts/ui/mission_complete_screen.gd

Game states:

- TITLE
- PLAYING
- PAUSED
- GAME_OVER
- MISSION_COMPLETE

Requirements:

- Start game from title screen
- Pause/unpause during gameplay
- Restart after game over
- Continue to next mission after mission complete, even if only one mission exists for now
- No gameplay movement while paused or in menus

Acceptance criteria:

- Game starts from title screen.
- Pause menu works.
- Game over screen works.
- Mission complete screen works.
- Restart returns to a playable state.

Phase 15 - Effects Pass

Implement Phase 15: effects pass.

Create:

- scenes/effects/Explosion.tscn
- scenes/effects/SmokeTrail.tscn
- scenes/effects/HitSpark.tscn
- scenes/effects/WaterSplash.tscn

- scripts/effects/explosion.gd
- scripts/effects/effects_manager.gd
- scripts/effects/screen_shake.gd

Effects:

- enemy death explosion
- bullet hit spark
- boss damage smoke
- aircraft water crash splash
- small screen shake on big explosions
- no excessive visual clutter

Use placeholders if no final particles/materials exist.

Acceptance criteria:

- Enemy death spawns explosion.
- Bullet hit spawns hit spark.
- Boss damage can show smoke.
- Effects clean themselves up or return to pool.
- Screen shake does not make gameplay unreadable.

Phase 16 - Audio Hooks

Implement Phase 16: audio manager and audio hooks.

Create:

- scripts/systems/audio_manager.gd
- placeholder references for:
 - player_fire
 - enemy_destroyed
 - player_hit
 - powerup_collect
 - boss_appear
 - mission_complete
 - game_over
 - menu_select
 - stage_music

Requirements:

- Use AudioStreamPlayer or AudioStreamPlayer3D appropriately.
- Do not require final audio assets.
- If placeholder audio files are missing, fail gracefully without errors.
- Add documentation explaining where to place audio files.

Acceptance criteria:

- Gameplay calls audio hooks.
- Missing audio does not crash the game.
- Volume can be controlled globally later.
- README explains audio asset placement.

Phase 17 - First Complete Stage

Implement Phase 17: first complete stage.

Create Stage 1: Open Pacific.

Stage content:

- scrolling ocean
- clouds
- at least 3 fighter waves
- at least 1 bomber wave
- at least 1 power-up drop
- enemy bullets
- boss encounter
- mission complete screen

Update:

- data/stages/stage_01.json
- data/waves/stage_01_waves.json
- docs/game_design.md

Stage duration target:

2 to 4 minutes for the prototype.

Acceptance criteria:

- Player can start from title screen.

- Player can play through Stage 1.
- Enemies spawn in waves.
- Player can die and get game over.
- Player can defeat boss and get mission complete.
- Score/lives/HUD work throughout.

Phase 18 - Ultra-Realistic Visual Direction Pass

Implement Phase 18: visual realism preparation.

Do not add heavy final assets yet unless they already exist.

Create documentation and placeholder material system for:

- realistic aircraft models
- ocean material
- cloud layers
- explosion particles
- smoke trails
- tracer bullets
- aircraft shadows
- water splashes
- ship wakes
- island chunks

Update:

- docs/art_direction.md
- docs/technical_plan.md
- assets/materials/README.md
- assets/models/README.md
- assets/textures/README.md

If useful, create simple placeholder materials:

- aircraft metal
- ocean blue
- smoke dark
- tracer glow
- explosion fire

Acceptance criteria:

- Project has clear asset replacement pipeline.
- Placeholder materials exist or are documented.
- Art direction explains top-down realistic style.
- Gameplay remains readable.

Phase 19 - Performance Optimization

Implement Phase 19: performance and cleanup pass.

Review the project for:

- excessive instantiation
- bullets not cleaning up
- effects not cleaning up
- unused scripts
- giant scripts that should be split
- repeated code
- weak signal connections
- hard-coded paths that should be exported variables
- missing error handling for JSON loading

Improve:

- object pooling
- cleanup logic
- JSON load validation
- scene references
- comments where helpful
- docs/known_issues.md

Do not change gameplay feel unless needed for a bug fix.

Acceptance criteria:

- Gameplay still works.
- No obvious memory leaks from bullets/effects.
- JSON loading has graceful error messages.
- docs/known_issues.md is updated.

Phase 20 - Export Preparation

Implement Phase 20: export preparation.

Create documentation for:

- how to export Windows build
- how to export Linux build
- how to export web build later
- how to export Android later
- how to replace placeholder assets
- how to test controls
- how to package the game

Create:

- docs/export_guide.md
- docs/testing_checklist.md
- docs/release_checklist.md

Add:

- credits template
- license note
- asset license tracking file

Do not assume final commercial release.

This is a prototype release package.

Acceptance criteria:

- Developer can follow docs/export_guide.md.
- testing_checklist.md covers core gameplay.
- release_checklist.md covers final manual checks.